

SC2006 Software Engineering

Lecture 12 Supplement

Conrad Watt

The Big Ideological Split in Software Engineering

Plan-driven methodologies

“If you fail to plan, you plan to fail”

- attributed to Benjamin Franklin

Agile methodologies

“A good plan violently executed now is better than a perfect plan next week”

- George Patton

The Big Ideological Split in Software Engineering

Plan-driven methodologies

Errors are more costly the later they're found (widely quoted - 30x)

So better to do extra planning early to catch more of these errors
("shift left" approach)

Agile methodologies

Overall development time is the biggest factor in software engineering costs

So better to avoid wasting time with too-detailed early plans that will likely be thrown out anyway

Just start writing code ASAP!

The Big Ideological Split in Software Engineering

Plan-driven methodologies

Can help break down a large project into clearly defined smaller problems

Help guide less-experienced developers

Could protect against a litigious client

Agile methodologies

Empower a small group of experienced developers to work quickly

Leave more leeway to react to unexpected changes in circumstance

The Big Ideological Split in Software Engineering

Plan-driven methodologies

May spend a lot of time on a plan only to discover an error or misconception during implementation

Last minute changes may require abandoning previous plans/diagrams or making time-consuming amendments

Agile methodologies

The final system may be a big mess:

- generally sloppy high-level design
- poor documentation
- missed opportunities for team members to share each other's work

How to participate?



1

Go to wooclap.com

2

Enter the event code in the top banner

Event code

KEKRRO

 [Copy participation link](#)

THE LIFE OF A SOFTWARE
ENGINEER.

CLEAN SLATE. SOLID
FOUNDATIONS. THIS TIME
I WILL BUILD THINGS THE
RIGHT WAY.



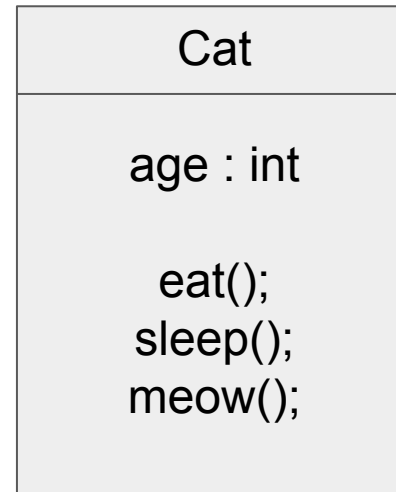
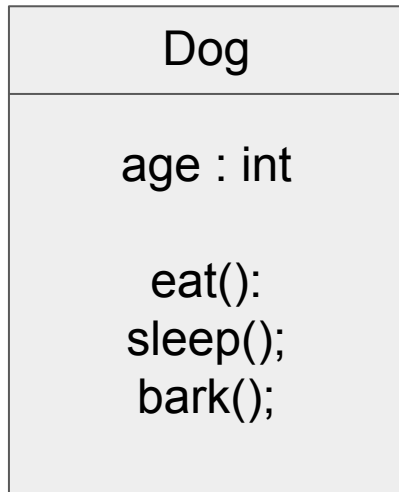
MUCH LATER...

OH MY. I'VE
DONE IT AGAIN,
HAVEN'T I ?

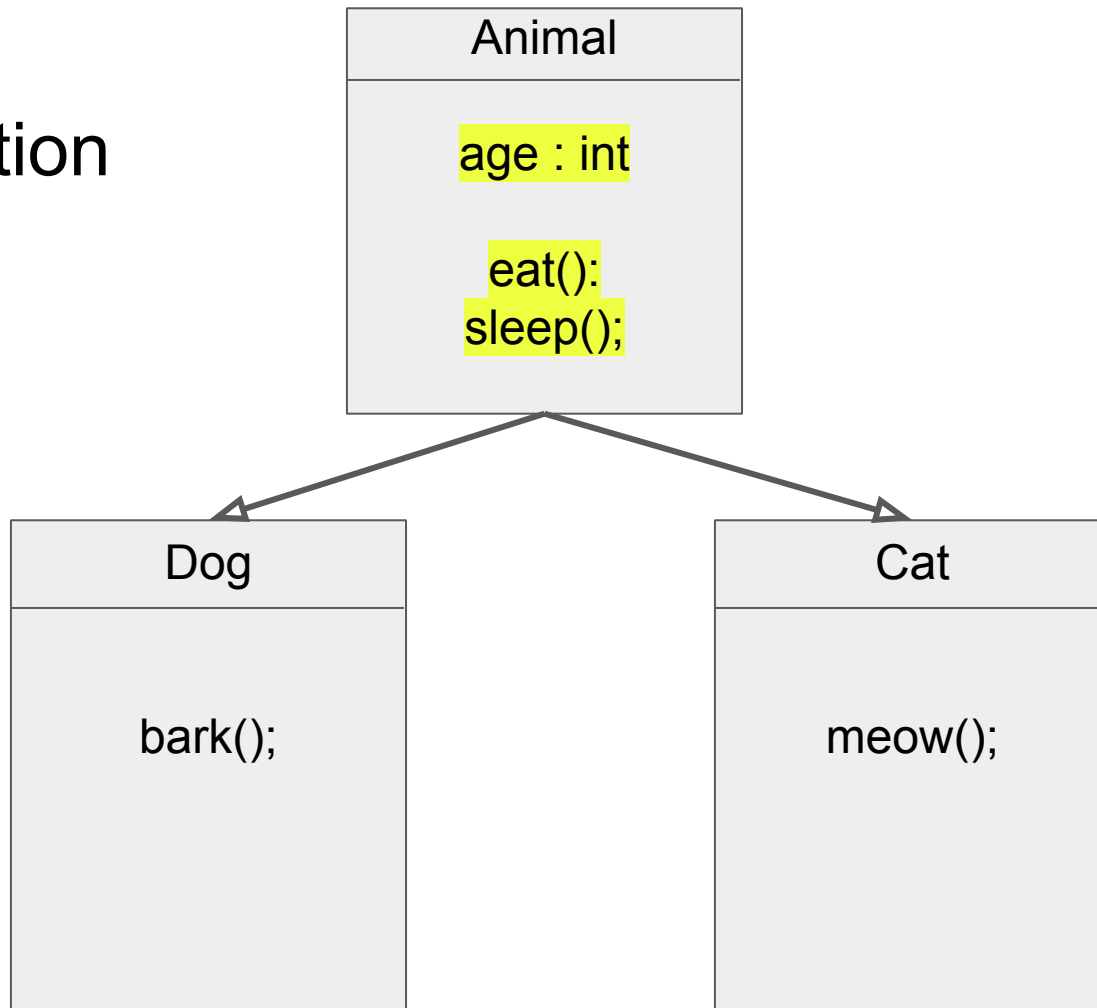


Abstraction

- Hiding or combining concrete details behind a more uniform interface



Abstraction



Abstraction

- A very general concept - not just about class hierarchies

`send_message_to_alice(String msg_text)`

`send_message_to_bob(String msg_text)`

Abstraction

- A very general concept - not just about class hierarchies

```
send_message_to_alice_or_bob(  
    String msg_text,  
    bool alice_or_bob)
```

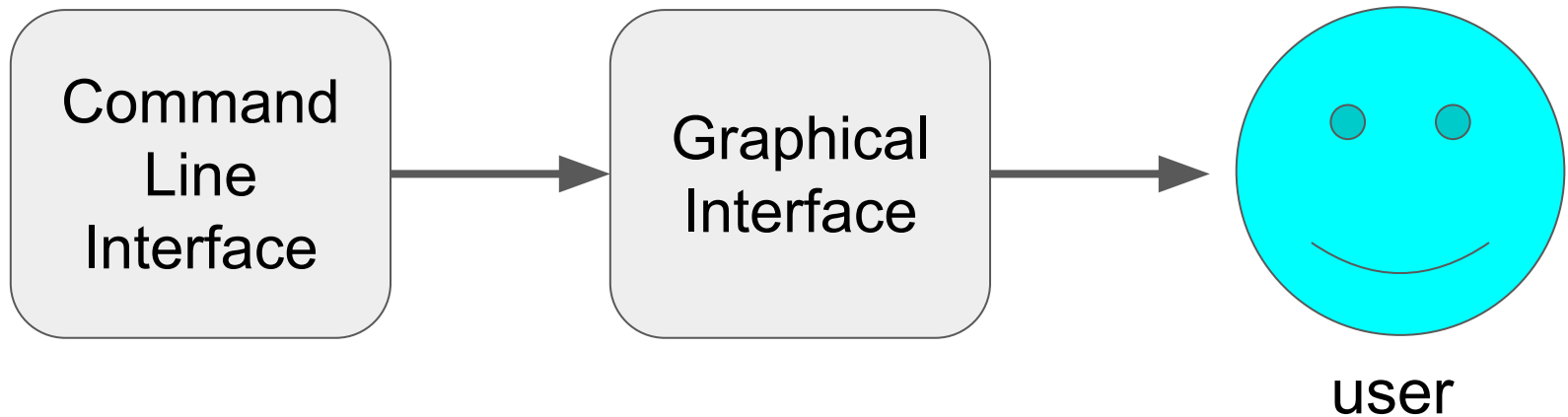
Abstraction

- A very general concept - not just about class hierarchies

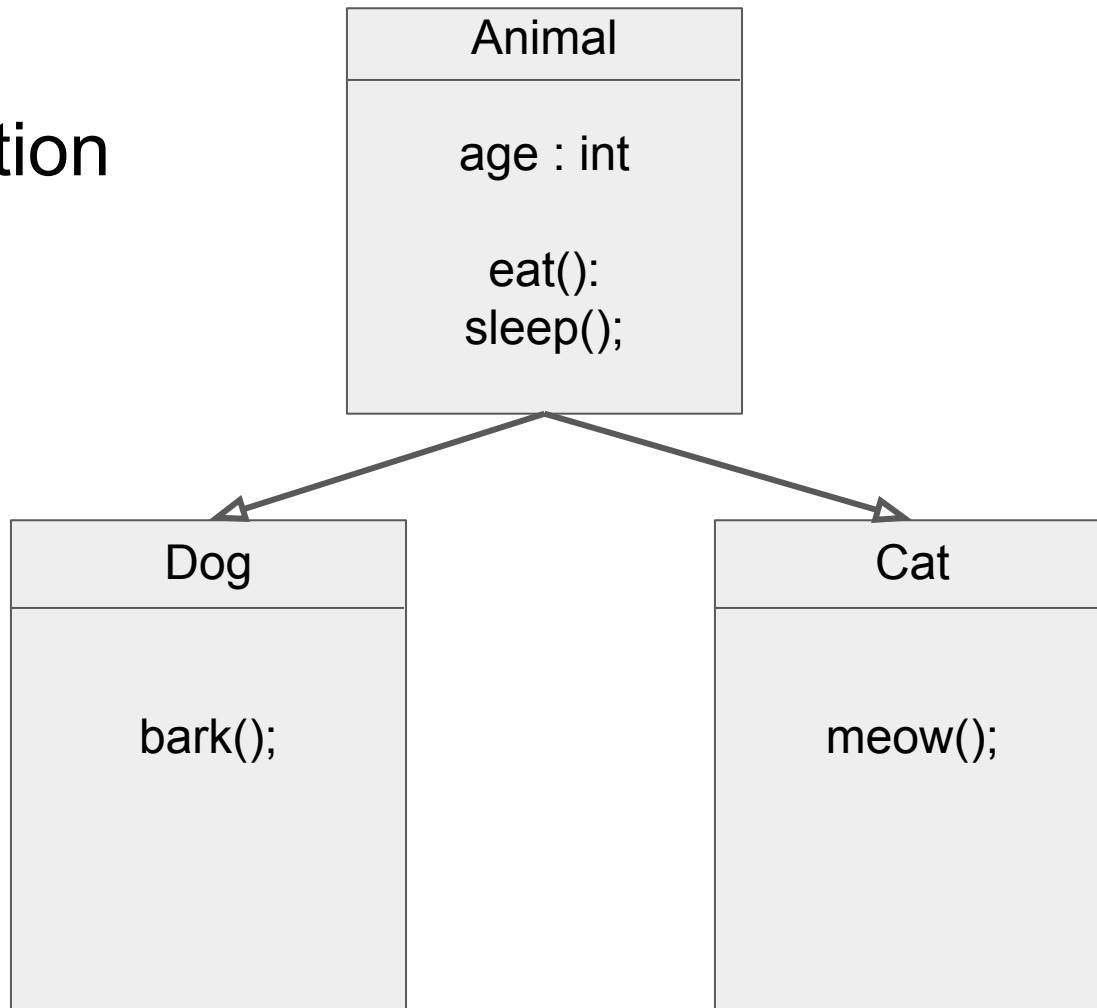
```
send_message_to_person(  
    String msg_text,  
    Person p)
```

Abstraction

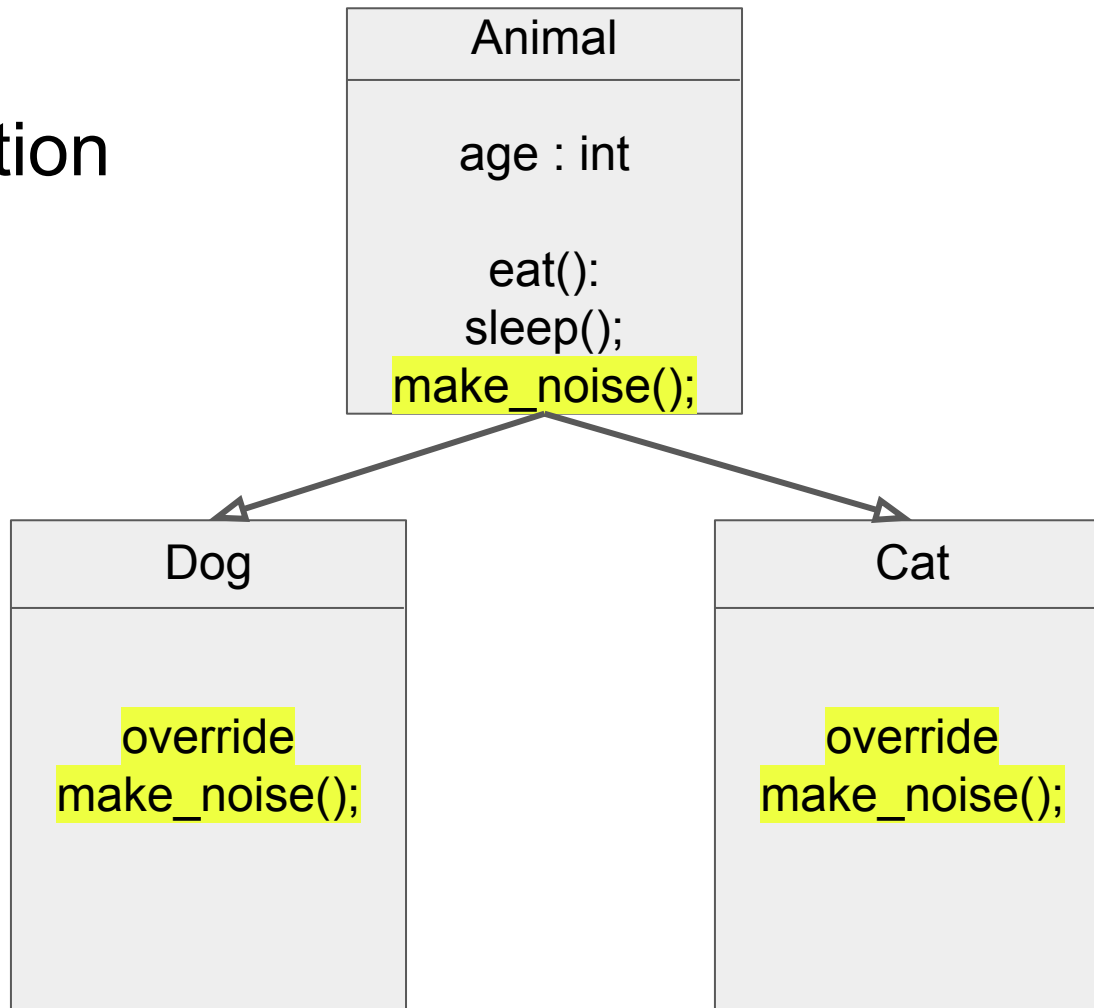
- A very general concept - not just about class hierarchies



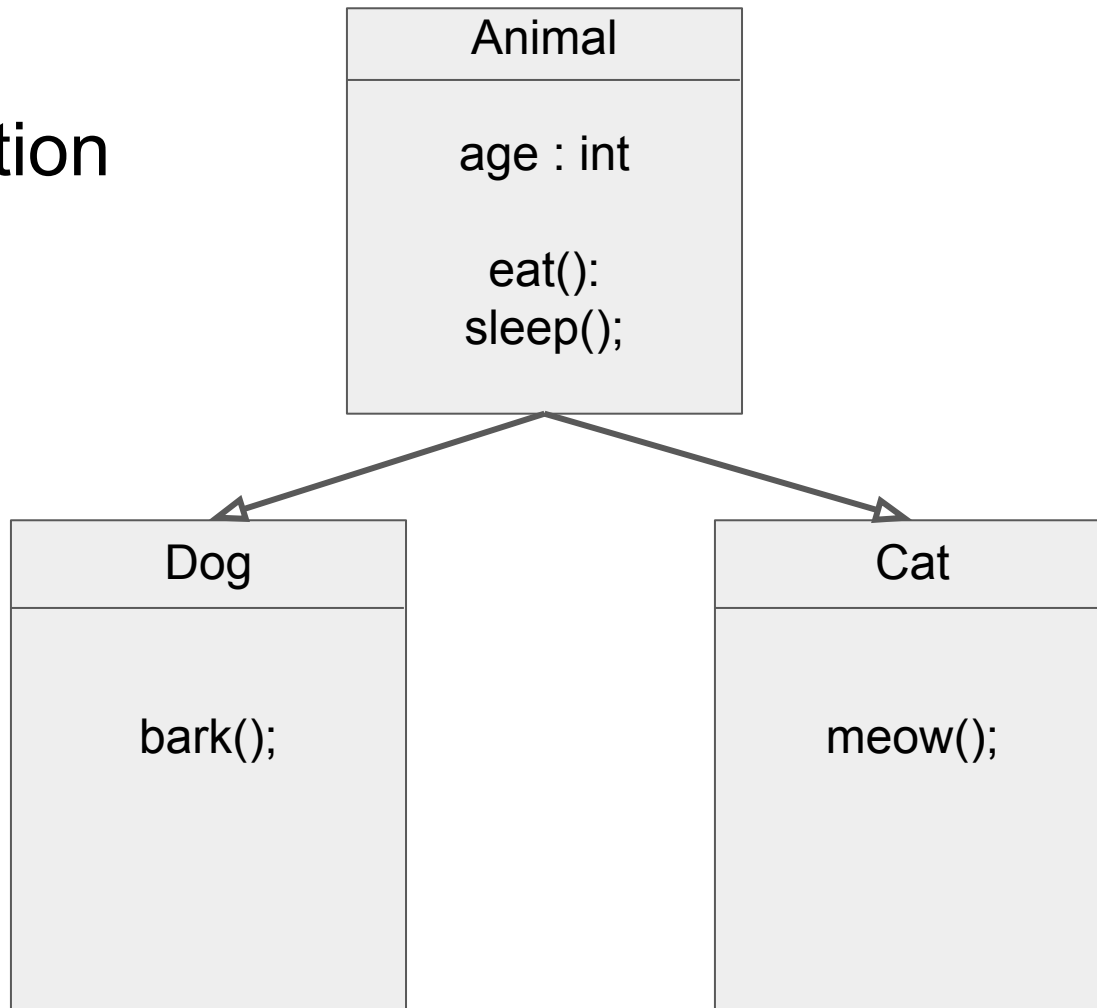
Abstraction



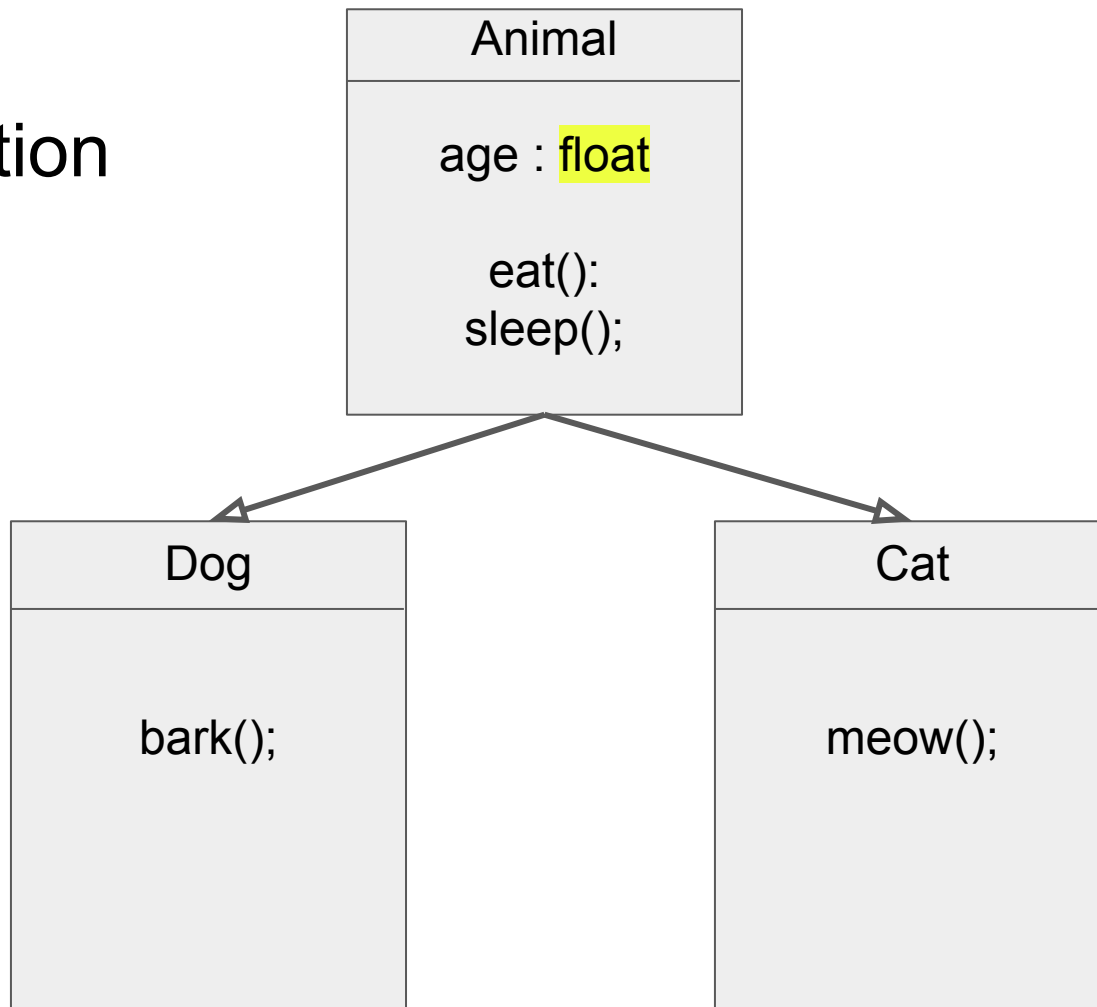
Abstraction



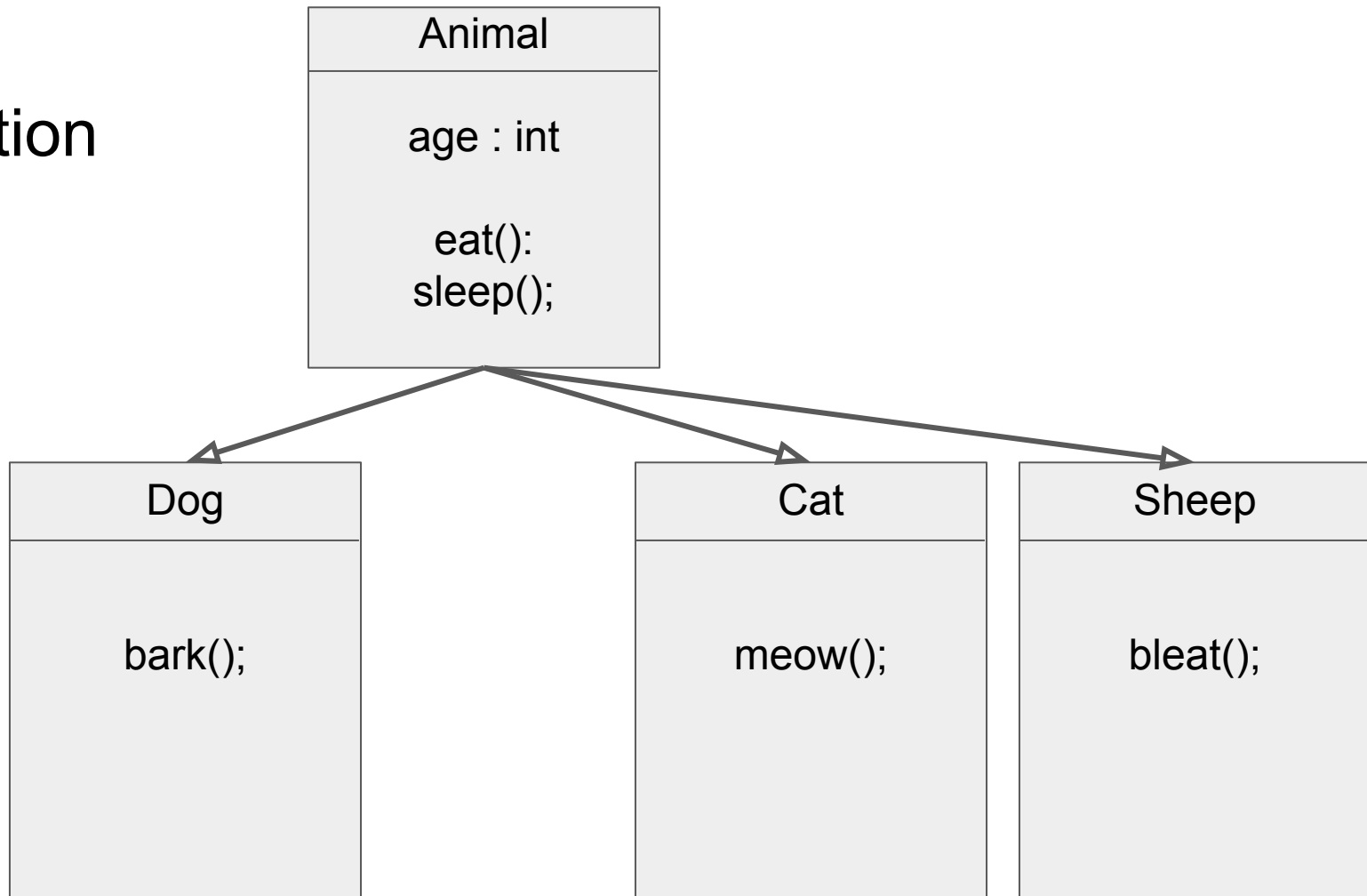
Abstraction



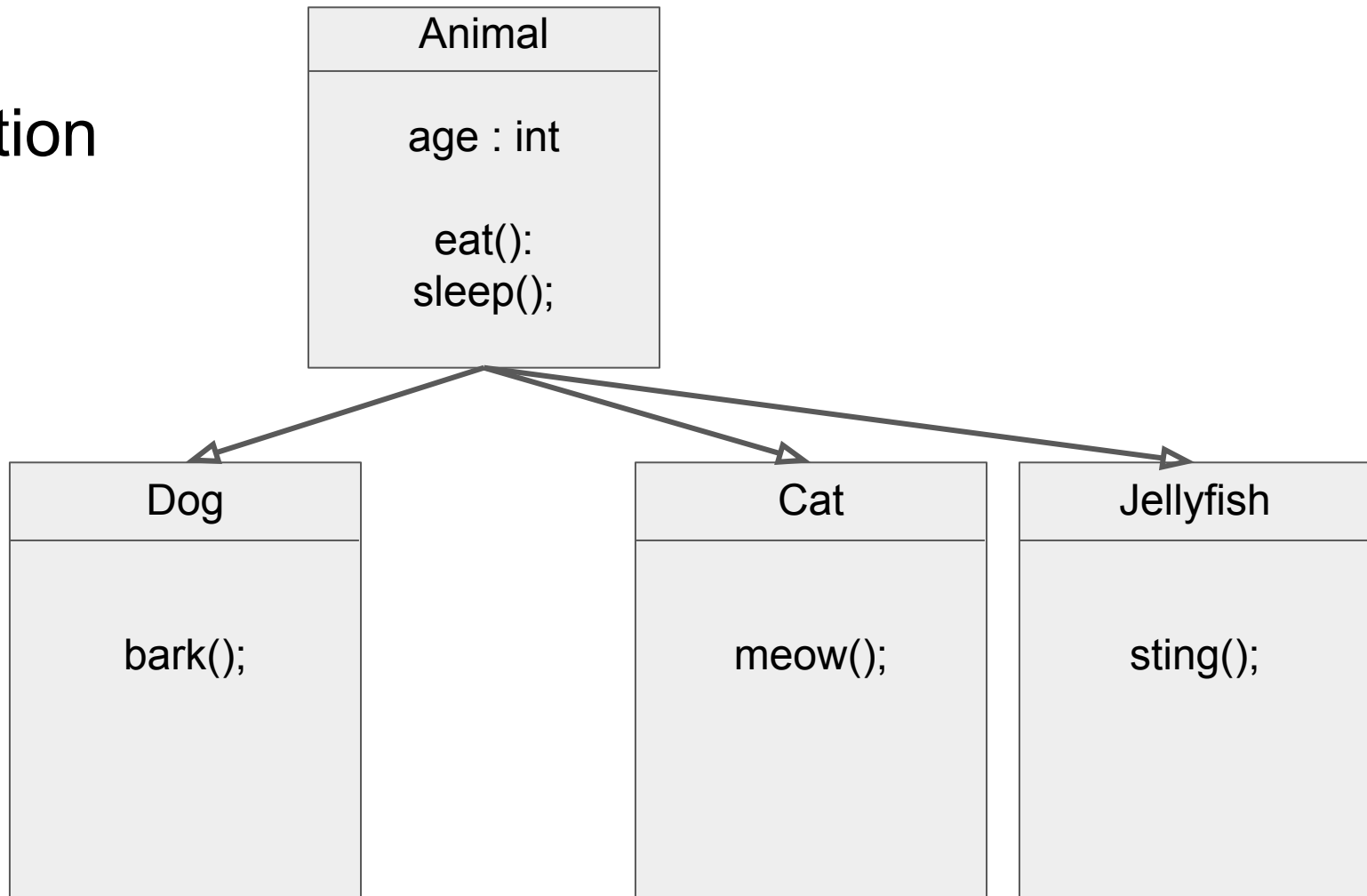
Abstraction



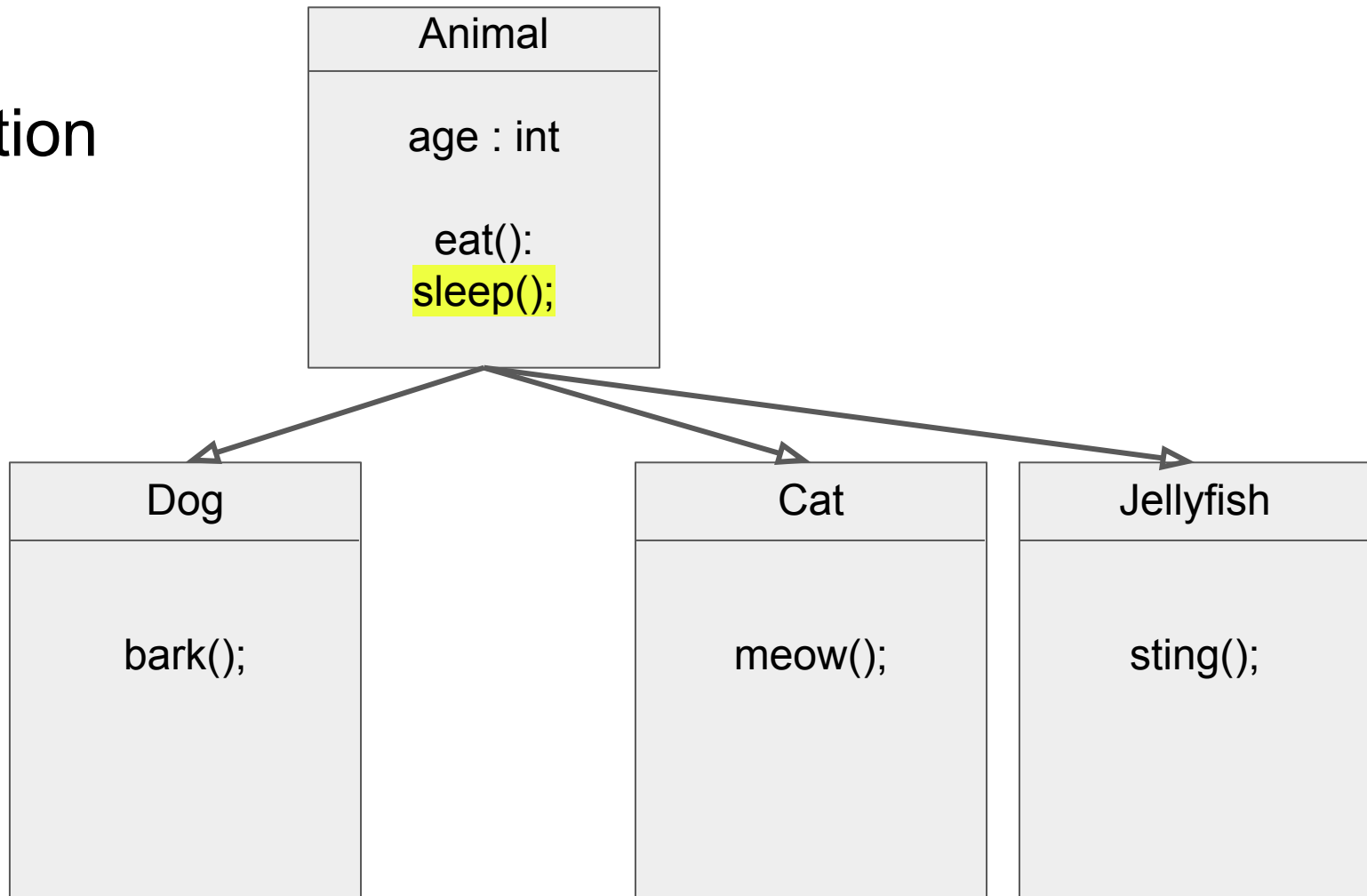
Abstraction



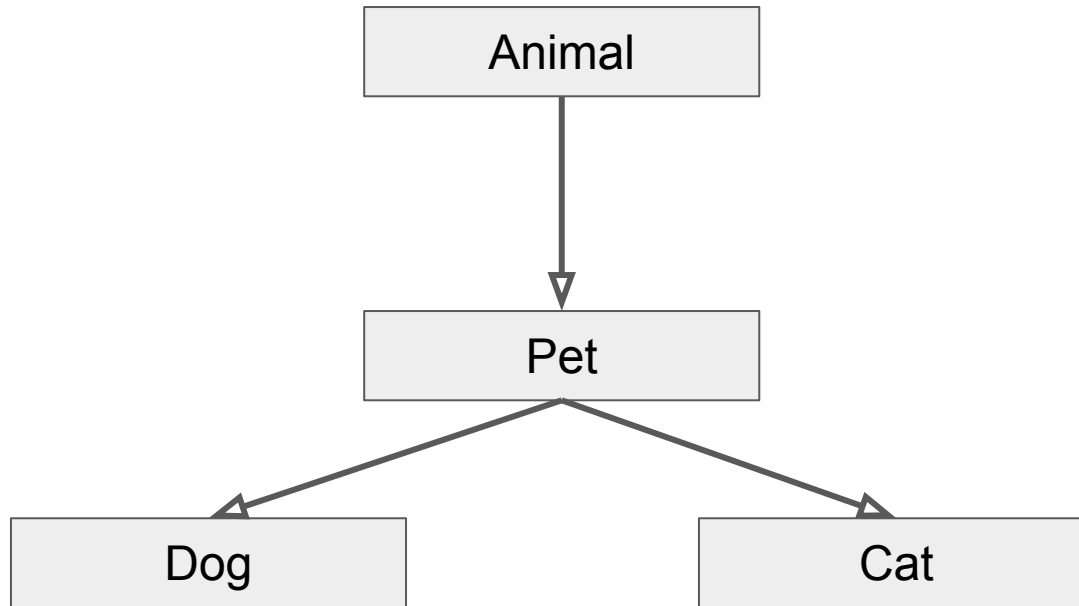
Abstraction



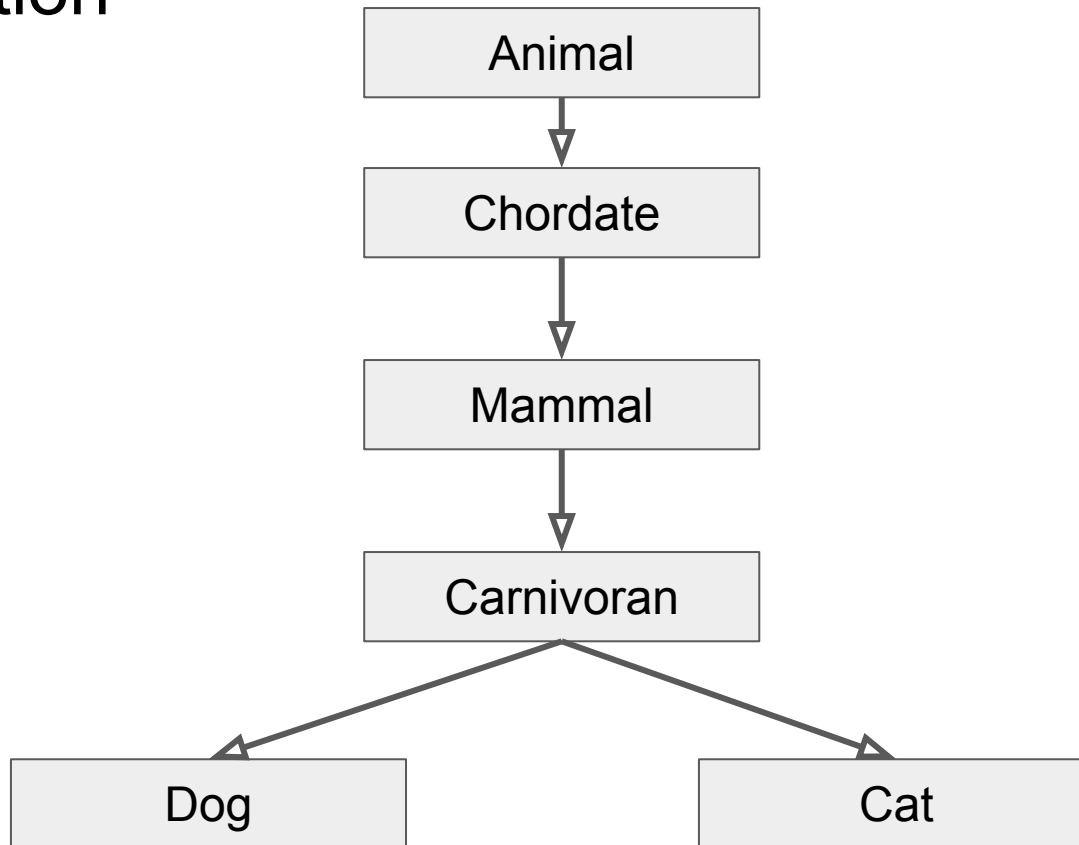
Abstraction



Abstraction



Abstraction



Abstraction



“The effective exploitation of... **abstraction must be regarded as one of the most vital activities of a **competent programmer.**”**

- Edsger Dijkstra

https://commons.wikimedia.org/wiki/File:Edsger_Wybe_Dijkstra.jpg

Hamilton Richards, CC BY-SA 3.0

Abstraction



https://upload.wikimedia.org/wikipedia/commons/a/a5/Donald_Ervin_Knuth_%28cropped%29.jpg

Alex Handy, CC BY-SA 2.0

**“Premature abstraction
is the root of all evil”**

- Donald Knuth